Proyecto ICA

Simulación de datos a partir de muestras

Carga de datos.

Realizamos carga de datos desde el archivo excel recibido del dpto de ing. quimica. Asimismo se da comienzo al código del proyecto importando bibliotecas necesarias

nota: este documento se exportó con `jupyter nbconvert --to html simulación.ipynb`

In [28]:
```
!pip install numpy scipy pandas matplotlib seaborn
```

```
5557.42s - pydevd: Sending message related to process being replaced timed-out after 5
seconds
Requirement already satisfied: numpy in ./.venv/lib/python3.8/site-packages (1.24.3)
Requirement already satisfied: scipy in ./.venv/lib/python3.8/site-packages (1.10.1)
Requirement already satisfied: pandas in ./.venv/lib/python3.8/site-packages (2.0.3)
Requirement already satisfied: matplotlib in ./.venv/lib/python3.8/site-packages (3.7.
3)
Requirement already satisfied: seaborn in ./.venv/lib/python3.8/site-packages (0.13.0)
Requirement already satisfied: python-dateutil>=2.8.2 in ./.venv/lib/python3.8/site-pa
ckages (from pandas) (2.8.2)
Requirement already satisfied: pytz>=2020.1 in ./.venv/lib/python3.8/site-packages (fr
om pandas) (2023.3.post1)
Requirement already satisfied: tzdata>=2022.1 in ./.venv/lib/python3.8/site-packages
(from pandas) (2023.3)
Requirement already satisfied: contourpy>=1.0.1 in ./.venv/lib/python3.8/site-packages
(from matplotlib) (1.1.1)
Requirement already satisfied: cycler>=0.10 in ./.venv/lib/python3.8/site-packages (fr
om matplotlib) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in ./.venv/lib/python3.8/site-package
s (from matplotlib) (4.42.1)
Requirement already satisfied: kiwisolver>=1.0.1 in ./.venv/lib/python3.8/site-package
s (from matplotlib) (1.4.5)
Requirement already satisfied: packaging>=20.0 in ./.venv/lib/python3.8/site-packages
(from matplotlib) (23.1)
Requirement already satisfied: pillow>=6.2.0 in ./.venv/lib/python3.8/site-packages (f
rom matplotlib) (10.0.1)
Requirement already satisfied: pyparsing>=2.3.1 in ./.venv/lib/python3.8/site-packages
(from matplotlib) (3.1.1)
Requirement already satisfied: importlib-resources>=3.2.0 in ./.venv/lib/python3.8/sit
e-packages (from matplotlib) (6.1.0)
Requirement already satisfied: zipp>=3.1.0 in ./.venv/lib/python3.8/site-packages (fro
m importlib-resources>=3.2.0->matplotlib) (3.17.0)
Requirement already satisfied: six>=1.5 in ./.venv/lib/python3.8/site-packages (from p
ython-dateutil>=2.8.2->pandas) (1.16.0)

[notice] A new release of pip is available: 23.2.1 -> 23.3
[notice] To update, run: pip install --upgrade pip
```

In [30]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import boxcox
from scipy.stats import yeojohnson


# Ruta al archivo de Excel
```

```
archivo_excel = '~/Escritorio/ProyectoICA/IIYNT-REQ-BD-001.xlsx'
# archivo_excel = '../../../Escritorio/ProyectoICA/IIYNT-REQ-BD-001.xlsx'

# Número de la fila desde la cual querés empezar (0-indexado)
fila_inicio = 3

# Leer el archivo Excel
init_df = pd.read_excel(archivo_excel)

# Mostrar las primeras filas del DataFrame para confirmar que se leyó bien
init_df.head()
```

Out[30]:

	Buscador por código de muestra	Rótulo de la muestra	Fecha (DD/MM/AAAA)	Horario (24 hrs XX:XX)	Ubicación	Temperatura Ambiental (°C)	Humedad	Tempe muest
0	260623ASA	A	2023-06-26 00:00:00	15:00:00	Arroyo Salguero	18.2	0.77	
1	Código de la muestra\n(fecha-ubicación-número)	Rótulo de la muestra	Fecha (DD/MM/AAAA)	Horario (24 hrs XX:XX)	Ubicación	Temperatura Ambiental (°C)	Humedad	Tempe de la m
2	030523ATA	A	2023-05-03 00:00:00	12:30:00	Arroyo_Las Torres	16	0.929	
3	090523PBA	A	2023-05-09 00:00:00	14:15:00	Puente Bilbao	17	0.47	
4	140623PBA	A	2023-06-14 00:00:00	14:30:00	Puente Bilbao	11.9	0.47	

5 rows × 23 columns

Transformación de Datos

Se seleccionan colúmnas válidas para la simulación

In [31]:

```
desired_columns = [
    'Temperatura Ambiental (°C)',
    'Humedad',
    'Temperatura de la muestra (°C)',
    'pH',
    'CE\n(µS/cm)',
    'STD\n(mg/L)',
    'STS\n(mL sed/L)',
    'OD\n(mg/L)',
    'Nivel (cm)',
    'Turbidez (NTU)',
    'Dureza\n(mg CaCO3/L)',
    'Cloruros\n(mg Cl-/L)'
]
df = init_df.iloc[2:102][desired_columns]
df.to_csv("dataset_limpio_para_chatGPT.csv")
```

Se realiza un control de consistencia de datos verificando que muestras tienen valores nulos

In [32]:

```
#df.isnull().sum()
df.isna().sum()
```

Out[32]:
```
Temperatura Ambiental (°C)         0
Humedad                            0
Temperatura de la muestra (°C)     0
pH                                 0
CE\n(µS/cm)                        0
STD\n(mg/L)                        0
STS\n(mL sed/L)                    3
OD\n(mg/L)                         0
Nivel (cm)                        40
Turbidez (NTU)                     2
Dureza\n(mg CaCO3/L)               1
Cloruros\n(mg Cl-/L)               3
dtype: int64
```

In [33]:
```
#Eliminar las filas que tienen valores nulos o no asignados
#df.dropna(inplace=True) #aca se modifica el Dataframe original con inplace=true

#Si asigno a nueva variable no hace falta inplace=true pero no modifica el Dataframe
#df_sin_nulos=df.dropna()

#Como opcion puedo llenar los valores nulos con datos adecuados:
#fillna(method='bfill') (llenar con el valor siguiente de la columna)
#fillna(method='ffill') (llenar con el valor anterior de la columna)
#fillna(0) (llenar con ceros)

mediaNivel=df['Nivel (cm)'].mean()
df['Nivel (cm)']=df['Nivel (cm)'].fillna(mediaNivel)

mediaSTS=df['STS\n(mL sed/L)'].mean()
df['STS\n(mL sed/L)']=df['STS\n(mL sed/L)'].fillna(mediaSTS)

mediaTurbidez=df['Turbidez (NTU)'].mean()
df['Turbidez (NTU)']=df['Turbidez (NTU)'].fillna(mediaTurbidez)

mediaDureza=df['Dureza\n(mg CaCO3/L)'].mean()
df['Dureza\n(mg CaCO3/L)']=df['Dureza\n(mg CaCO3/L)'].fillna(mediaDureza)

mediaCloruros=df['Cloruros\n(mg Cl-/L)'].mean()
df['Cloruros\n(mg Cl-/L)']=df['Cloruros\n(mg Cl-/L)'].fillna(mediaCloruros)

df_sin_nulos=df
df_sin_nulos.isna().sum()

#Para eliminar columnas con valores nulos uso:
#df_sin_columnas_con_nulos=df.dropna(axis=1)
```

Out[33]:
```
Temperatura Ambiental (°C)         0
Humedad                            0
Temperatura de la muestra (°C)     0
pH                                 0
CE\n(µS/cm)                        0
STD\n(mg/L)                        0
STS\n(mL sed/L)                    0
OD\n(mg/L)                         0
Nivel (cm)                         0
Turbidez (NTU)                     0
Dureza\n(mg CaCO3/L)               0
Cloruros\n(mg Cl-/L)               0
dtype: int64
```

Sección de simulación.

Se utilizarán varias tecnicas de simulación y se expondrán las ventajas y problemas de cada una a continuación. Si bien utilizo la simulación de Montecarlo para expandir los datos de manera

independiente, ésta da diferentes problemas de inconsistencia (como turbidez negativa) que intento afrontar de difersas maneras Estas maneras son:

1. **Simulación por defecto**: Se expondrán resultados con inconsistencias arrojadas para visualizar el problema

2. **Truncar en valores maximos y minimos**: Cualquier valor fuera del rango es truncado a su contorno

3. **Truncar en Cero**: Cualquier valor negativo se convierte en cero. Es el enfoque más simple, pero puede introducir sesgo en tus simulaciones.

4. **Distribuciones No Negativas**: Usar una distribución que solo genere valores no negativos, como la distribución log-normal o la exponencial. Pero esto puede complicar la interpretación y el ajuste de los datos.

5. **Re-simulación**: Cada vez que se obtiene un valor negativo, se vuelve a simular hasta obtener un valor positivo. Puede ser computacionalmente intensivo si hay muchas re-simulaciones.

6. **Transformación de Datos**: A veces, una transformación (como tomar el logaritmo) hace que los datos se ajusten mejor a una distribución normal. Luego puedes aplicar la transformación inversa a los datos simulados.

   6.1. **Transformación de Box-Cox**

   6.2. **Transformación de Yeo-Johnson**

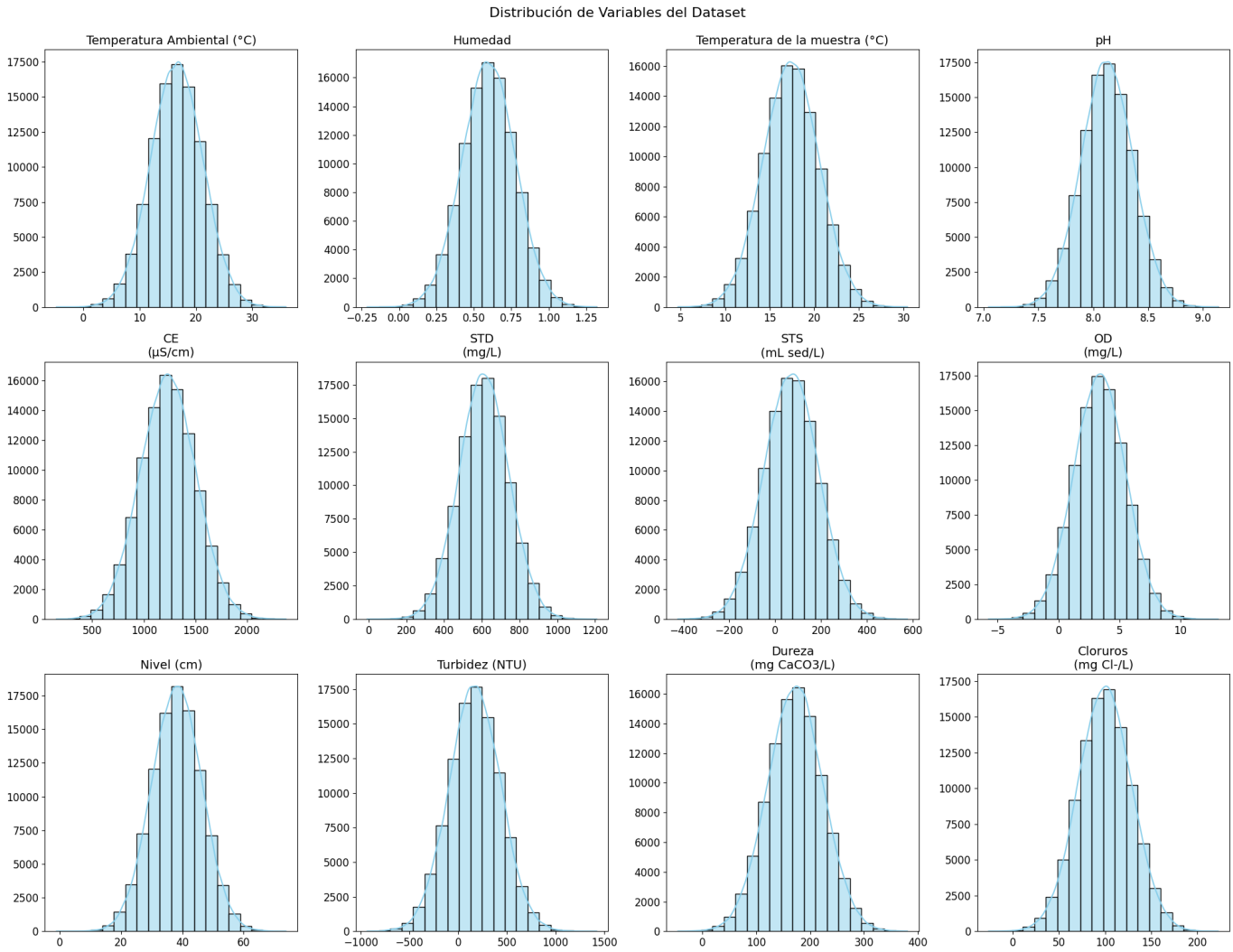
(A continuación, se definen funciones de apoyo)

In [34]:
```
def plot_results(df: pd.DataFrame) -> None:
    """
    Función que grafíca el histograma de cada columna del dataframe
    """
    # Determine the number of columns in the data
    n_cols = len(df.columns)

    # Calculate the number of rows needed for the subplots
    n_rows = int(np.ceil(n_cols / 4))

    # Create a figure and a grid of subplots
    fig, axes = plt.subplots(nrows=n_rows, ncols=4, figsize=(20, 5 * n_rows))

    # Flatten the array of axes for easier indexing
    axes_flat = axes.flatten() if n_rows > 1 else axes

    # Remove any excess subplots
    for i in range(n_cols, len(axes_flat)):
        fig.delaxes(axes_flat[i])

    # Create histograms
    for i, col in enumerate(df.columns):
        ax = axes_flat[i]
        sns.histplot(df[col], bins=20, color='skyblue', kde=True, ax=ax)
        ax.set_title(f'{col}', fontsize=14)
        ax.set_xlabel('')
        ax.set_ylabel('')
        ax.tick_params(axis='x', labelsize=12)
        ax.tick_params(axis='y', labelsize=12)

    # Improve the layout
```

```
    plt.tight_layout()
    plt.suptitle('Distribución de Variables del Dataset', fontsize=16, y=1.02)
    plt.show()
```

Simulación 1, Simulación por defecto

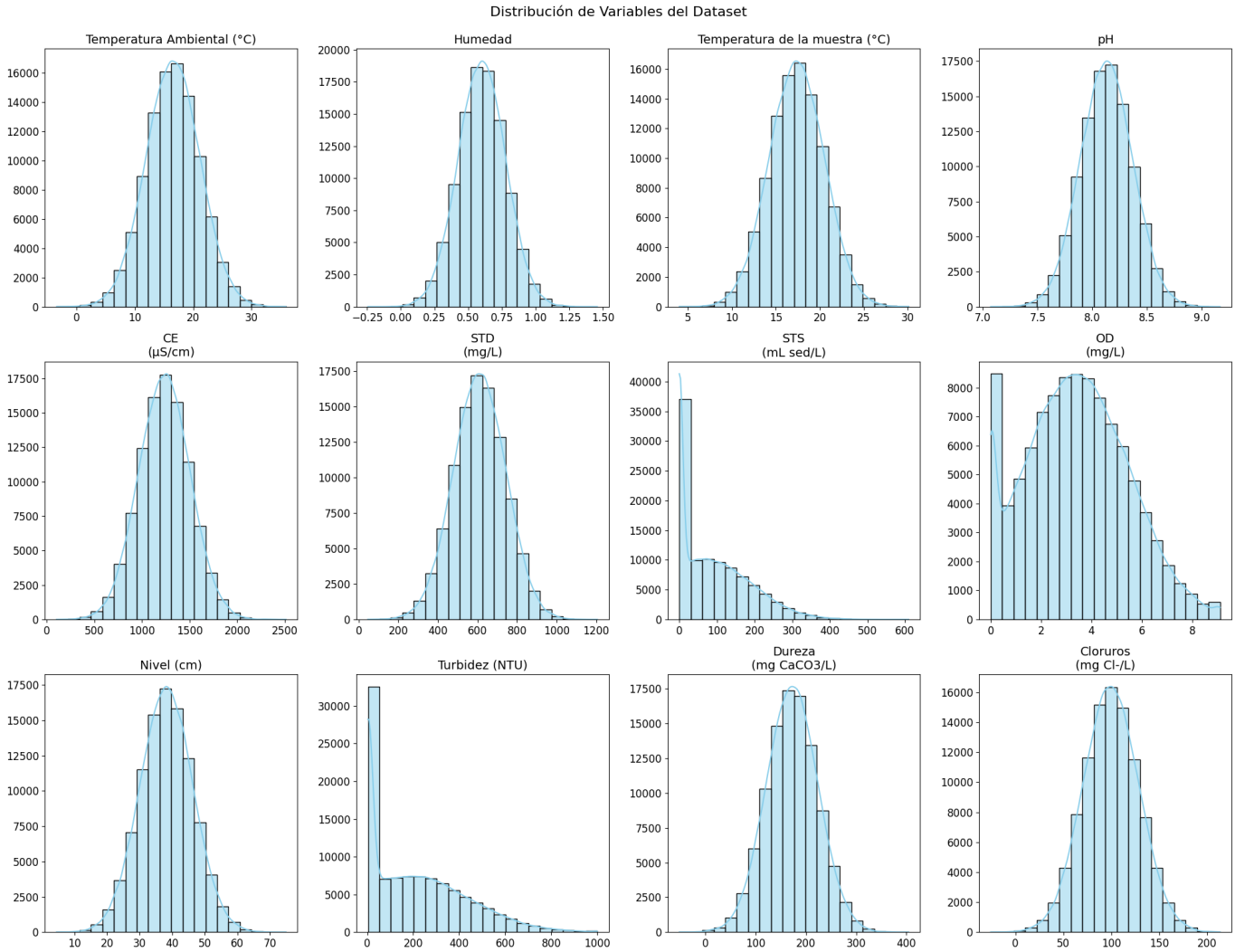
In [35]:
```
# Leé el CSV original para no modificar df_sin_nulos por las dudas
df_original = df_sin_nulos.copy()

# Creá un DataFrame vacío para guardar los resultados
df_simulado = pd.DataFrame()

# Número de simulaciones que querés hacer
num_simulaciones = 100000 #Si quiero tener en cuenta las relaciones entre las variabl


for columna in df_original.columns:
    # Calculá la media y la desviación estándar de cada columna
    media = df_original[columna].mean()
    std = df_original[columna].std()

    # Generá datos aleatorios siguiendo una distribución normal
    datos_simulados = np.random.normal(media, std, num_simulaciones)

    # Añadí estos datos al DataFrame de resultados
    df_simulado[columna] = datos_simulados

df_simulado.to_csv("./dataset_simulado_opcion_1.csv")
plot_results(df_simulado)
```



Simulación 2, truncar en valores maximos y minimos

In [24]:
```
# Leé el CSV original para no modificar df_sin_nulos por las dudas
df_original = df_sin_nulos.copy()

# Creá un DataFrame vacío para guardar los resultados
df_simulado = pd.DataFrame()

# Número de simulaciones que querés hacer
num_simulaciones = 100000 #Si quiero tener en cuenta las relaciones entre las variabl


for columna in df_original.columns:
    # Calculá la media y la desviación estándar de cada columna
    media = df_original[columna].mean()
    std = df_original[columna].std()

    # Generá datos aleatorios siguiendo una distribución normal
    datos_simulados = np.random.normal(media, std, num_simulaciones)

    # Añadí estos datos al DataFrame de resultados
    df_simulado[columna] = datos_simulados



""" for i, valor in enumerate(archivo_excel['Turbidez (NTU)']): #esto si fuera un exc
    if valor<val_min_STS:val_max_turbidez
       archivo_excel.at[i,'Turbidez (NTU)']=val_min_STS
    if valor>val_max_STS:
       archivo_excel.at[i,'Turbidez (NTU)']=val_max_STS """


#aplico correcciones a la columna STS con el metodo apply que aplica una funcion lamb
val_min_STS=0.1
val_max_STS=650
df_simulado['STS\n(mL sed/L)'] = df_simulado['STS\n(mL sed/L)'].apply(
    lambda x: val_min_STS if x < val_min_STS else (val_max_STS if x > val_max_STS els
)

#aplico correcciones a la columna OD con el metodo apply que aplica una funcion lambd
val_min_OD=0.0
val_max_OD=9.1
df_simulado['OD\n(mg/L)']=df_simulado['OD\n(mg/L)'].apply(
    lambda x: val_min_OD if x < val_min_OD else (val_max_OD if x > val_max_OD else x)
)

#aplico correcciones a la columna turbidez con el metodo apply que aplica una funcion
val_min_turbidez=2.7
val_max_turbidez=1000.0
df_simulado['Turbidez (NTU)']=df_simulado['Turbidez (NTU)'].apply(
    lambda x: val_min_turbidez if x < val_min_turbidez else (val_max_turbidez if x >
)


#Si quiero tener en cuenta las relaciones entre las variables planteando escenarios:

#Para STD: Es casi exacto no planteo escenarios
# df_simulado['STD\n(mg/L)'] = -0.4958 + 0.4934 * df_simulado['CE\n(µS/cm)']

# #Para Cloruros:
# #Regresion lineal simple
# df_simulado['Cloruros\n(mg Cl-/L)'] = -52.32 + 0.1265 * df_simulado['CE\n(µS/cm)']
# #Escenarios:
# def escenarioCl(valor):
#     probabilidad=np.random.uniform(0,1)
#     if probabilidad <= 0.3:
#         return valor * 0.93
#     elif probabilidad >=0.7:
```

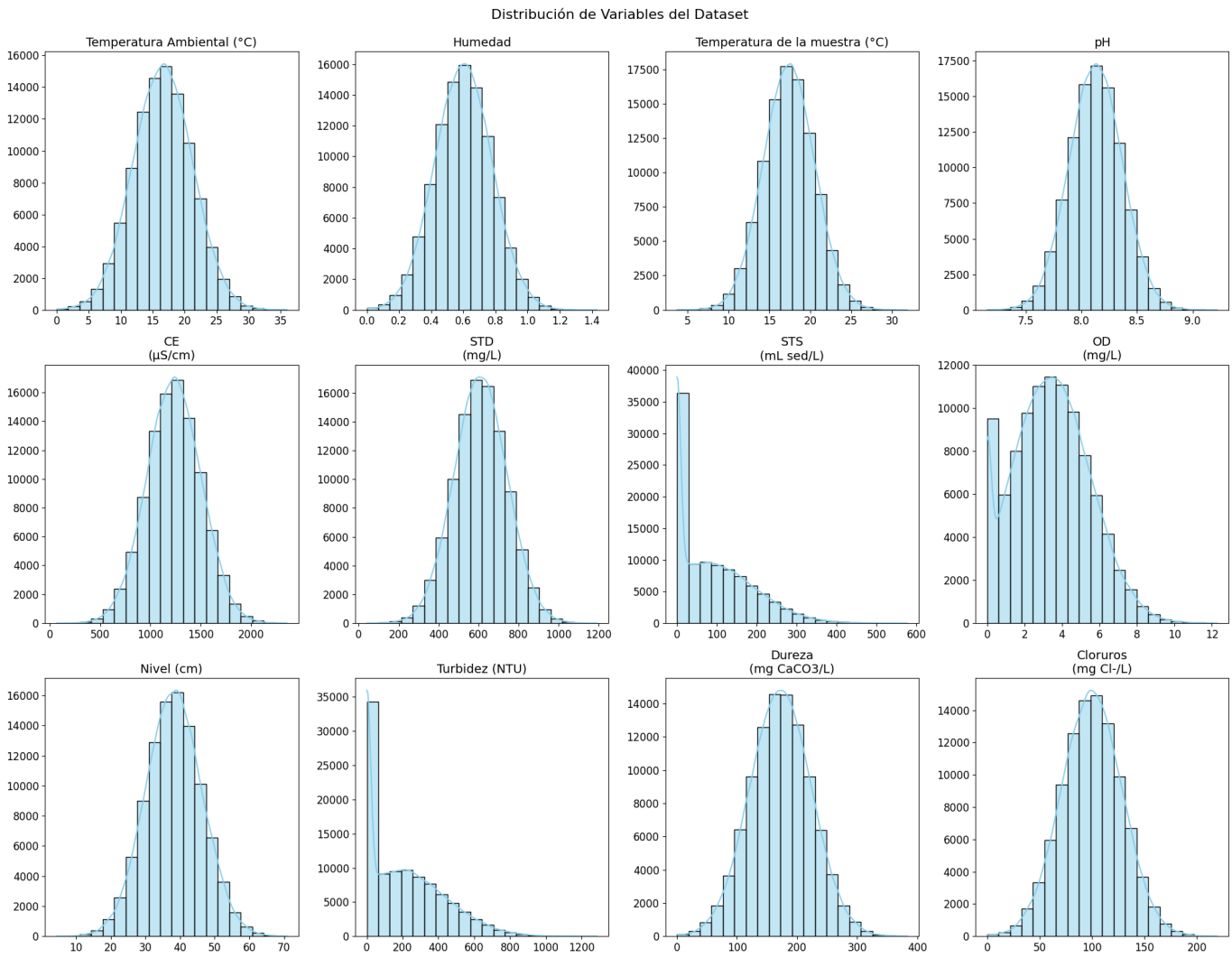
```
#         return valor * 1.07
#     else:
#         return valor

# df_simulado['Cloruros\n(mg Cl-/L)'] = df_simulado['Cloruros\n(mg Cl-/L)'].apply(esc

# val_min_cloruros=15.0
# val_max_cloruros=156.0
# df_simulado['Cloruros\n(mg Cl-/L)']=df_simulado['Cloruros\n(mg Cl-/L)'].apply(lambd


# Guardá el DataFrame simulado como CSV
df_simulado.to_csv("./dataset_simulado_opcion_2.csv", index=False) #../
plot_results(df_simulado)
```



Simulación 3, truncar en Cero

In [25]:
```
# Leé el CSV original para no modificar df_sin_nulos por las dudas
df_original = df_sin_nulos.copy()

# Creá un DataFrame vacío para guardar los resultados
df_simulado = pd.DataFrame()

# Número de simulaciones que querés hacer
num_simulaciones = 100000 #Si quiero tener en cuenta las relaciones entre las variabl


for columna in df_original.columns:
    # Calculá la media y la desviación estándar de cada columna
    media = df_original[columna].mean()
    std = df_original[columna].std()

    # Generá datos aleatorios siguiendo una distribución normal
```

```
    datos_simulados = np.random.normal(media, std, num_simulaciones)

    # Añadí estos datos al DataFrame de resultados
    df_simulado[columna] = datos_simulados

# Truncate negative values to zero
df_simulado_truncado = df_simulado.apply(lambda x: np.where(x < 0, 0, x))
plot_results(df_simulado_truncado)
df_simulado_truncado.to_csv("./dataset_simulado_opcion_3.csv", index=False) #../
```

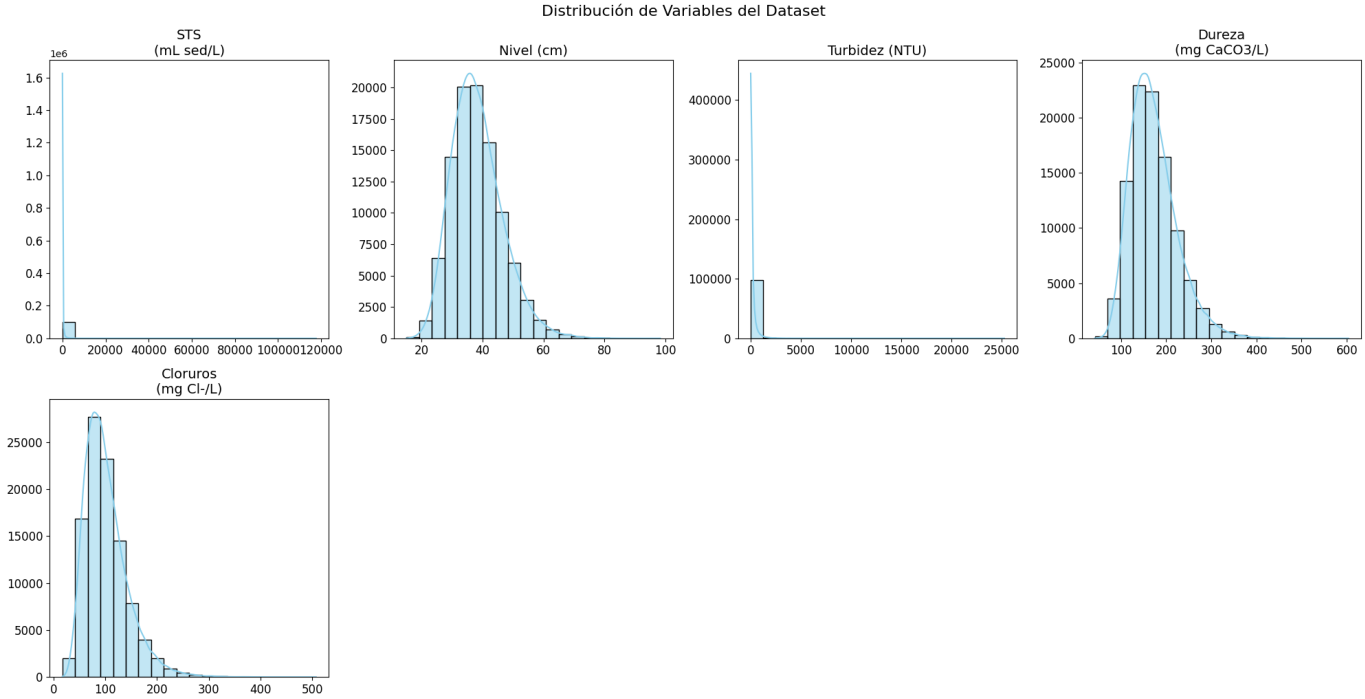


Simulación 4, distribuciones No Negativas

Simulación 4.1, Distribucion Lognormal

In [26]:
```
df_original = df_sin_nulos.copy()

# Create an empty DataFrame to store the log-normally distributed simulated data
df_simulado_lognormal = pd.DataFrame()

# Generate simulated data for each column using log-normal distribution
for col in df_original.columns:
    if df_original[col].dtype == 'float64':  # We only simulate for numerical columns
        # Remove zeros to avoid issues with log transformation
        non_zero_data = df_original[df_original[col] > 0][col]

        if len(non_zero_data) > 0:
            # Calculate the mean and standard deviation of the log-transformed data
            mu_log = np.log(non_zero_data).mean()
            sigma_log = np.log(non_zero_data).std()

            # Generate log-normal distributed random numbers
            lognormal_data = np.random.lognormal(mu_log, sigma_log, num_simulaciones)
```

```
            df_simulado_lognormal[col] = lognormal_data

# Calculate the mean and standard deviation for the log-normally distributed simulate
mean_values_lognormal = df_simulado_lognormal.mean()
std_values_lognormal = df_simulado_lognormal.std()

plot_results(df_simulado_lognormal)
df_simulado_lognormal.to_csv("./dataset_simulado_distribucion_lognormal.csv", index=F
```



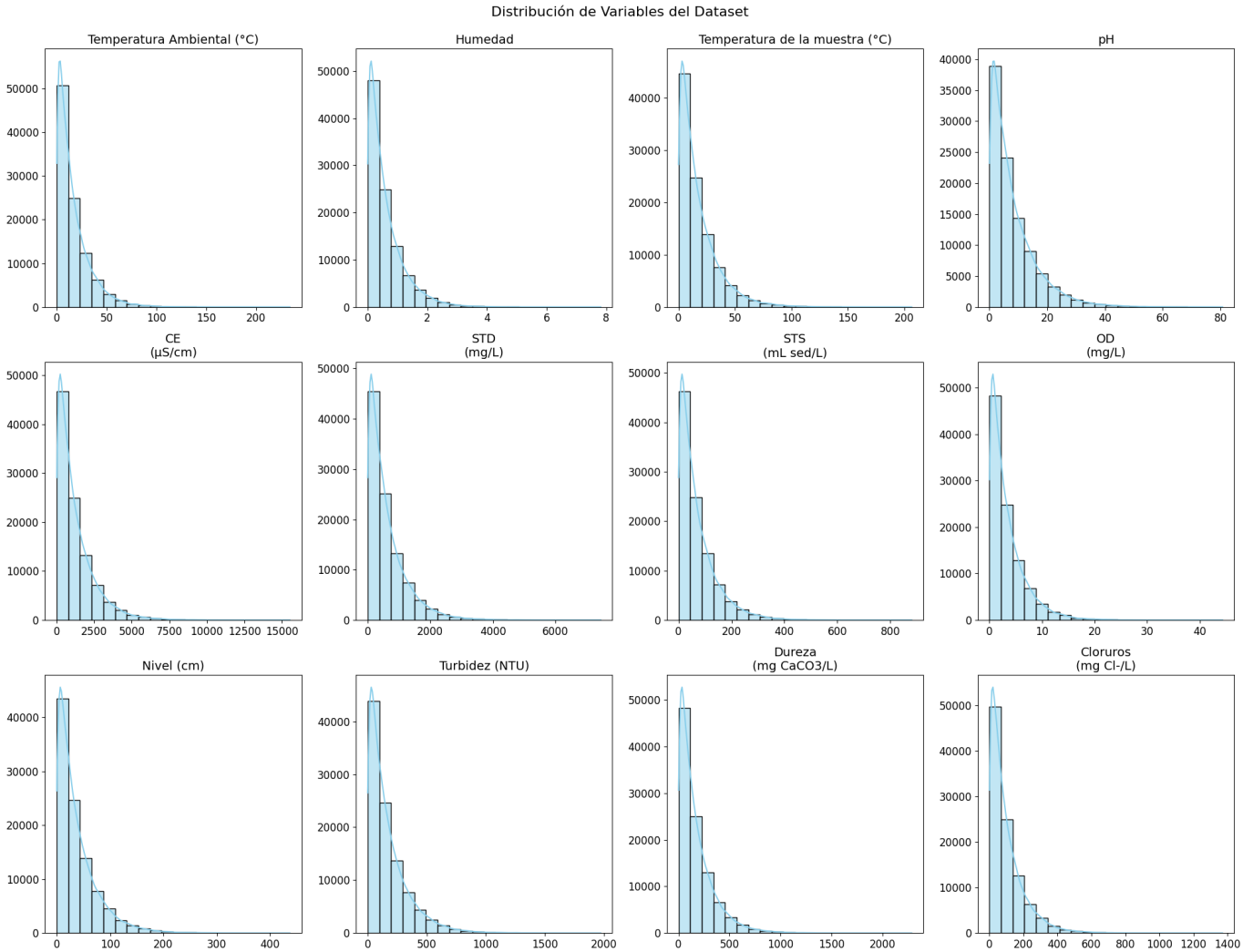
Nota al pié: La distribución log-normal suele funcionar bien para variables que son inherentemente no negativas y que tienen un sesgo hacia valores más grandes. Sin embargo, no todas las variables en tu conjunto de datos pueden ser buenos candidatos para una distribución log-normal desde un punto de vista físico o químico. Por ejemplo:

- **Temperatura Ambiental (°C) y Temperatura de la Muestra (°C)**: Estas podrían ajustarse con una log-normal, pero en muchos casos una distribución normal podría ser más adecuada, especialmente si las temperaturas pueden fluctuar en ambos lados de una media.
- **Humedad y pH**: Estos son parámetros que ya tienen límites físicos (por ejemplo, el pH varía generalmente entre 0 y 14), y en muchos casos pueden estar bien descritos por una distribución normal en lugar de una log-normal.
- **STS (mL sed/L) y Turbidez (NTU)**: Estos parámetros describen la cantidad de partículas en el agua y podrían ajustarse bien a una log-normal, especialmente si hay una alta variabilidad y los valores son siempre positivos.
- **OD (mg/L)**: El oxígeno disuelto es un parámetro que también podría seguir una distribución log-normal, especialmente si hay una tendencia hacia valores bajos que no pueden ser negativos.
- **Nivel (cm), Dureza (mg CaCO3/L) y Cloruros (mg Cl-/L)**: Estos son parámetros que también podrían ajustarse bien con una log-normal, dado que son inherentemente no negativos.

En resumen, el ajuste depende mucho del contexto específico y del rango de variabilidad de cada parámetro

Simulación 4.2, Distribucion Exponencial

In [27]:
```python
# Leé el CSV original para no modificar df_sin_nulos por las dudas
df_original = df_sin_nulos.copy()

# Creá un DataFrame vacío para guardar los resultados
df_simulado = pd.DataFrame()

# Número de simulaciones que querés hacer
num_simulaciones = 100000

for columna in df_original.columns:
    # Calculá la media de cada columna
    media = df_original[columna].mean()

    # Generá datos aleatorios siguiendo una distribución exponencial
    datos_simulados = np.random.exponential(scale=media, size=num_simulaciones)

    # Añadí estos datos al DataFrame de resultados
    df_simulado[columna] = datos_simulados

df_simulado.to_csv("./dataset_simulado_distribucion_exponencial.csv")
plot_results(df_simulado)
```



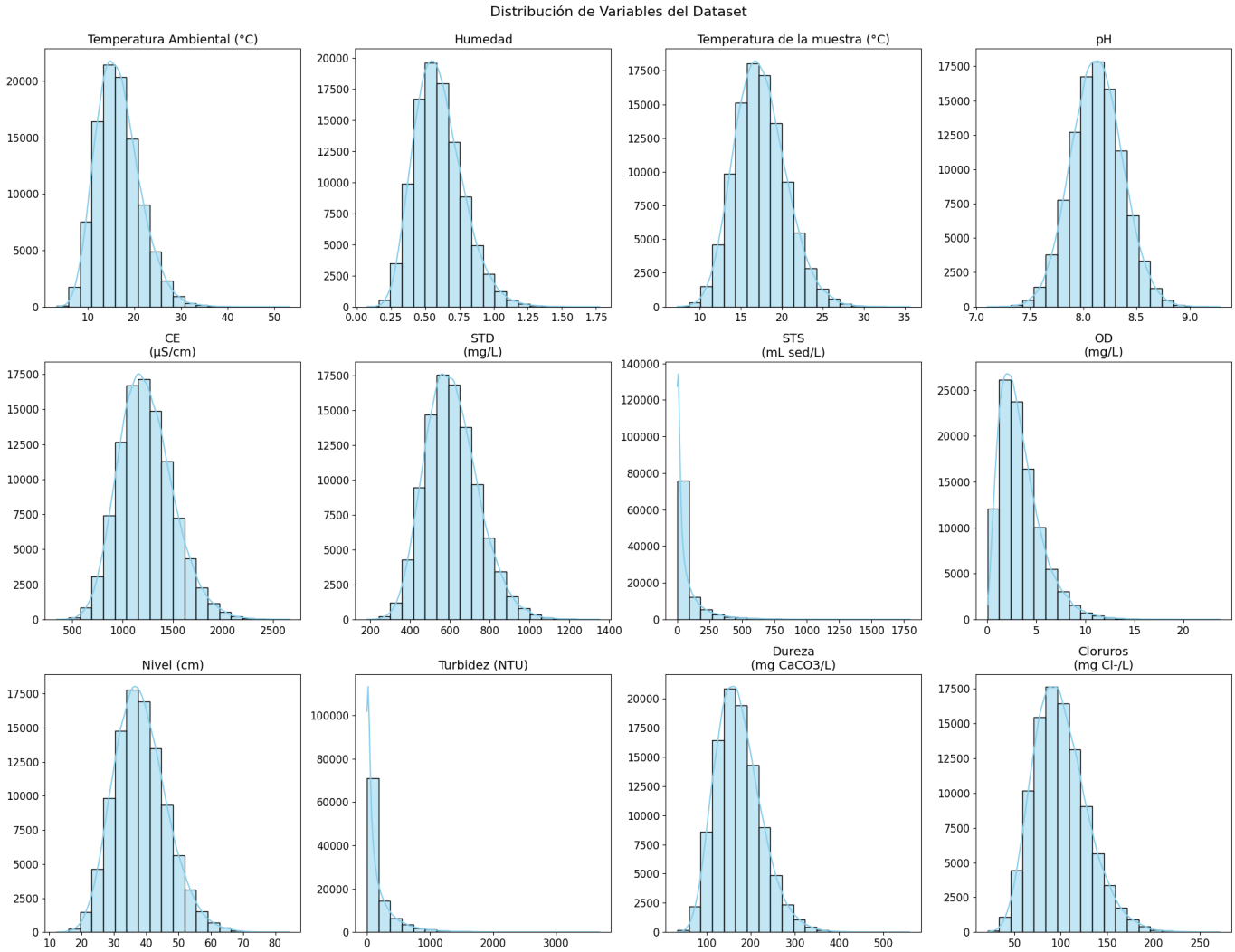
Distribución Exponencial:

La distribución exponencial es útil para modelar tiempos entre eventos sucesivos en procesos de Poisson, y siempre produce valores positivos.

Simulación 4.2, Distribucion Gamma

In [29]:
```python
# Leé el CSV original para no modificar df_sin_nulos por las dudas
df_original = df_sin_nulos.copy()

# Creá un DataFrame vacío para guardar los resultados
```

```python
df_simulado = pd.DataFrame()

# Número de simulaciones que querés hacer
num_simulaciones = 100000

for columna in df_original.columns:
    # Calculá la media y la desviación estándar de cada columna
    media = df_original[columna].mean()
    std = df_original[columna].std()

    # Parámetros de la distribución gamma (shape y scale)
    shape = (media / std) ** 2
    scale = std ** 2 / media

    # Generá datos aleatorios siguiendo una distribución gamma
    datos_simulados = np.random.gamma(shape, scale, num_simulaciones)

    # Añadí estos datos al DataFrame de resultados
    df_simulado[columna] = datos_simulados

df_simulado.to_csv("./dataset_simulado_distribucion_gamma.csv")
plot_results(df_simulado)
```



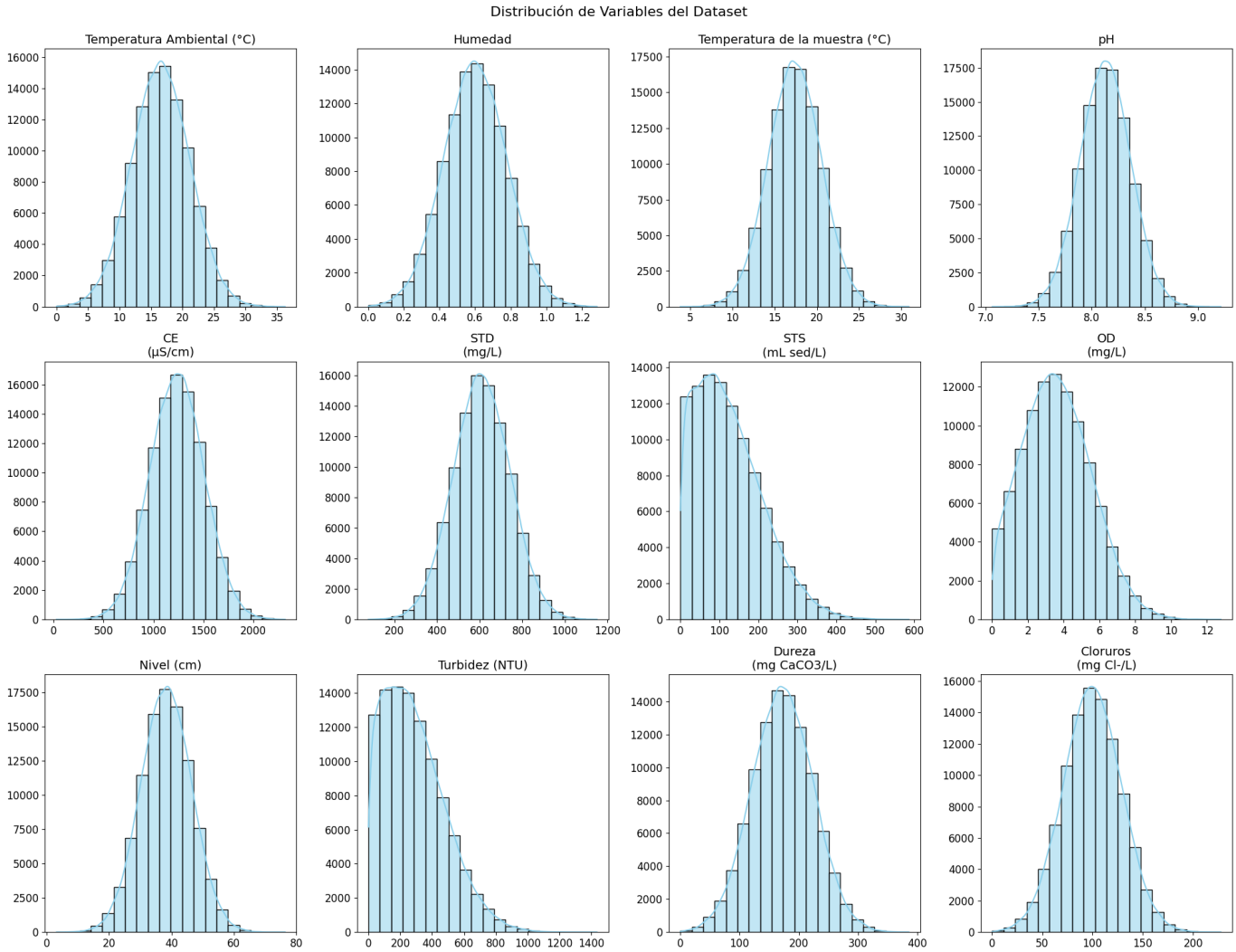
Distribución Gamma:

La distribución gamma es una generalización de la distribución exponencial y puede ajustarse a una variedad de datos positivos.

Simulación 5, re-simulación

In [11]:
```python
# Copia del DataFrame sin nulos para la simulación
df_original = df_sin_nulos.copy()
```

```
# Calculamos la media y la desviación estándar del DataFrame original
media_original = df_original.mean()
desviacion_original = df_original.std()

# Creamos un DataFrame vacío para guardar los datos re-simulados
df_simulado_resimulado = pd.DataFrame()

# Generamos datos re-simulados para cada columna
for columna in df_original.columns:
    media_columna = media_original[columna]  # Media
    desviacion_columna = desviacion_original[columna]  # Desviación estándar
    datos_simulados = np.random.normal(media_columna, desviacion_columna, num_simulac

    # Re-simulamos cualquier valor negativo
    while np.any(datos_simulados < 0):
        datos_simulados[datos_simulados < 0] = np.random.normal(media_columna, desvia

    df_simulado_resimulado[columna] = datos_simulados

plot_results(df_simulado_resimulado)
df_simulado_resimulado.to_csv("./dataset_simulado_opcion_5.csv", index=False)
```



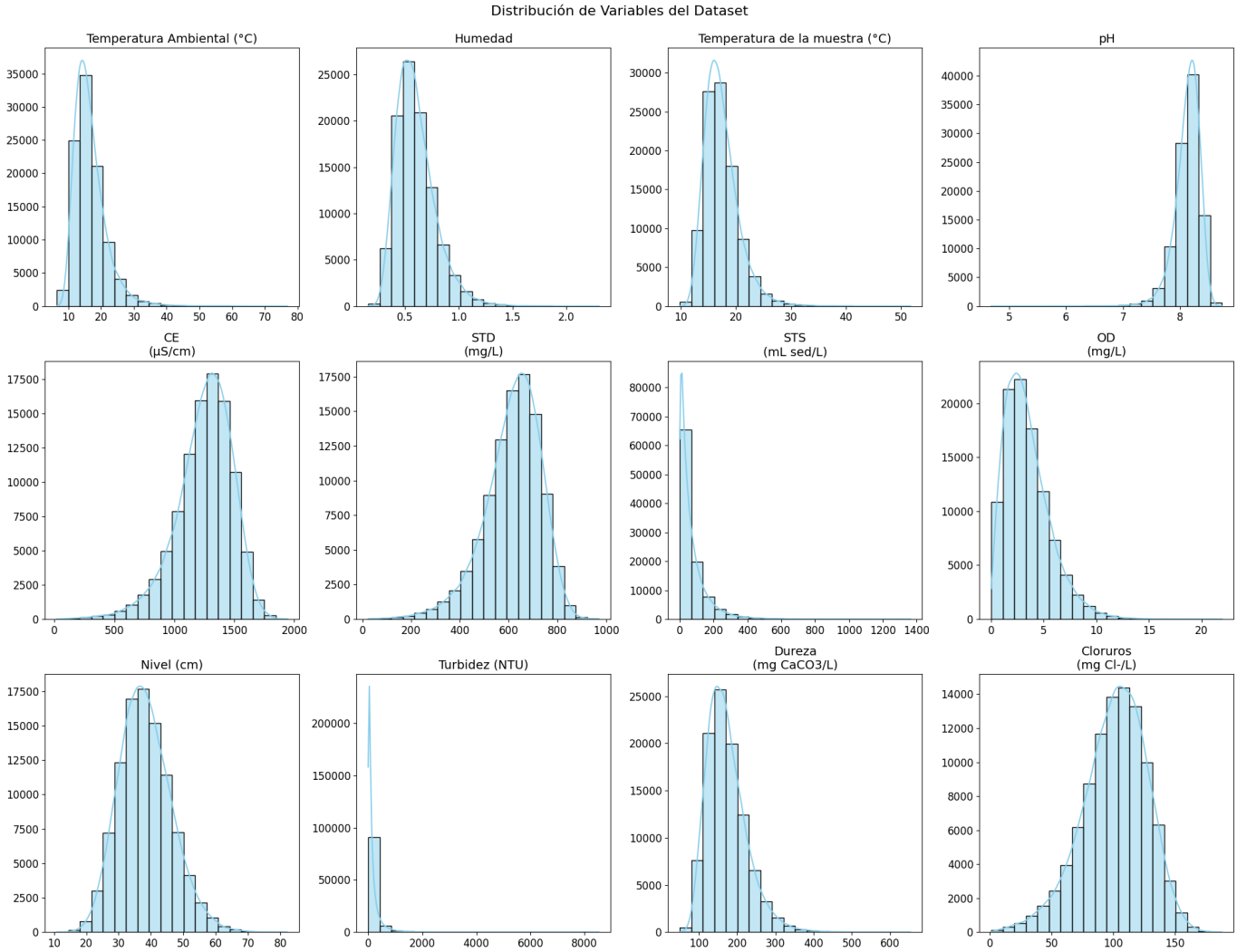
Simulaciones 6. Tranformaciones:

Simulación 6.1. , Transformación de Box-Cox

In [12]:
```
# Copia del DataFrame sin nulos para la simulación
df_original = df_sin_nulos.copy().astype('float64')



# Creamos un DataFrame vacío para guardar los datos transformados con Box-Cox
df_simulado_boxcox = pd.DataFrame()
```

```python
# Generamos datos simulados para cada columna usando la transformación Box-Cox
for columna in df_original.columns:

    # Eliminamos los ceros para evitar problemas con la transformación Box-Cox
    datos_sin_cero = df_original[df_original[columna] > 0][columna]

    if len(datos_sin_cero) > 0:
        # Aplicamos la transformación Box-Cox
        datos_transformados, valor_lambda = boxcox(datos_sin_cero)

        # Calculamos la media y la desviación estándar de los datos transformados
        media_transformada = np.mean(datos_transformados)
        desviacion_transformada = np.std(datos_transformados)

        # Generamos números aleatorios con distribución normal
        datos_normales = np.random.normal(media_transformada, desviacion_transformada

        # Aplicamos la transformación inversa de Box-Cox
        if valor_lambda == 0:
            datos_simulados = np.exp(datos_normales)
        else:
            datos_simulados = (valor_lambda * datos_normales + 1) ** (1 / valor_lambd

        df_simulado_boxcox[columna] = datos_simulados

plot_results(df_simulado_boxcox)
df_simulado_boxcox.to_csv("./dataset_simulado_boxcox.csv", index=False)
```

```
/tmp/ipykernel_6688/393213518.py:30: RuntimeWarning: invalid value encountered in powe
r
  datos_simulados = (valor_lambda * datos_normales + 1) ** (1 / valor_lambda)
/tmp/ipykernel_6688/393213518.py:30: RuntimeWarning: invalid value encountered in powe
r
  datos_simulados = (valor_lambda * datos_normales + 1) ** (1 / valor_lambda)
/tmp/ipykernel_6688/393213518.py:30: RuntimeWarning: invalid value encountered in powe
r
  datos_simulados = (valor_lambda * datos_normales + 1) ** (1 / valor_lambda)
/tmp/ipykernel_6688/393213518.py:30: RuntimeWarning: invalid value encountered in powe
r
  datos_simulados = (valor_lambda * datos_normales + 1) ** (1 / valor_lambda)
/tmp/ipykernel_6688/393213518.py:30: RuntimeWarning: invalid value encountered in powe
r
  datos_simulados = (valor_lambda * datos_normales + 1) ** (1 / valor_lambda)
/tmp/ipykernel_6688/393213518.py:30: RuntimeWarning: invalid value encountered in powe
r
  datos_simulados = (valor_lambda * datos_normales + 1) ** (1 / valor_lambda)
```



Explicación de la transformación Box-Cox

La transformación Box-Cox es una familia de transformaciones potenciales que se utilizan para estabilizar la varianza y hacer que un dataset se ajuste mejor a una distribución normal. La fórmula general de la transformación Box-Cox para un conjunto de datos (x) y un parámetro (\lambda) es la siguiente:

[y(x) =

$$\begin{cases} \frac{x^{\lambda}-1}{\lambda} & \text{si } \lambda \neq 0 \\ \ln(x) & \text{si } \lambda = 0 \end{cases}$$

]

Simulación de Datos

1. **Datos originales sin ceros**: Primero, se eliminan los ceros del conjunto de datos original para cada columna, ya que la transformación Box-Cox no se define para (x \leq 0).

2. **Aplicar Box-Cox**: Luego, se aplica la transformación Box-Cox a estos datos para obtener un conjunto transformado. Esta transformación se hace para que los datos transformados se ajusten mejor a una distribución normal.

3. **Calcular estadísticas**: Se calculan la media y la desviación estándar de los datos transformados.

4. **Generar datos aleatorios**: Se generan datos aleatorios con una distribución normal usando la media y la desviación estándar calculadas. Estos datos son una simulación de los datos

transformados.

Transformación Inversa de Box-Cox

Una vez que se tienen los datos simulados, se aplica la transformación inversa de Box-Cox para llevarlos de vuelta al espacio original. La fórmula de la transformación inversa es:

[x(y) =

$$\begin{cases} (e^y) & \text{si } \lambda = 0 \\ ((\lambda y + 1)^{1/\lambda}) & \text{si } \lambda \neq 0 \end{cases}$$

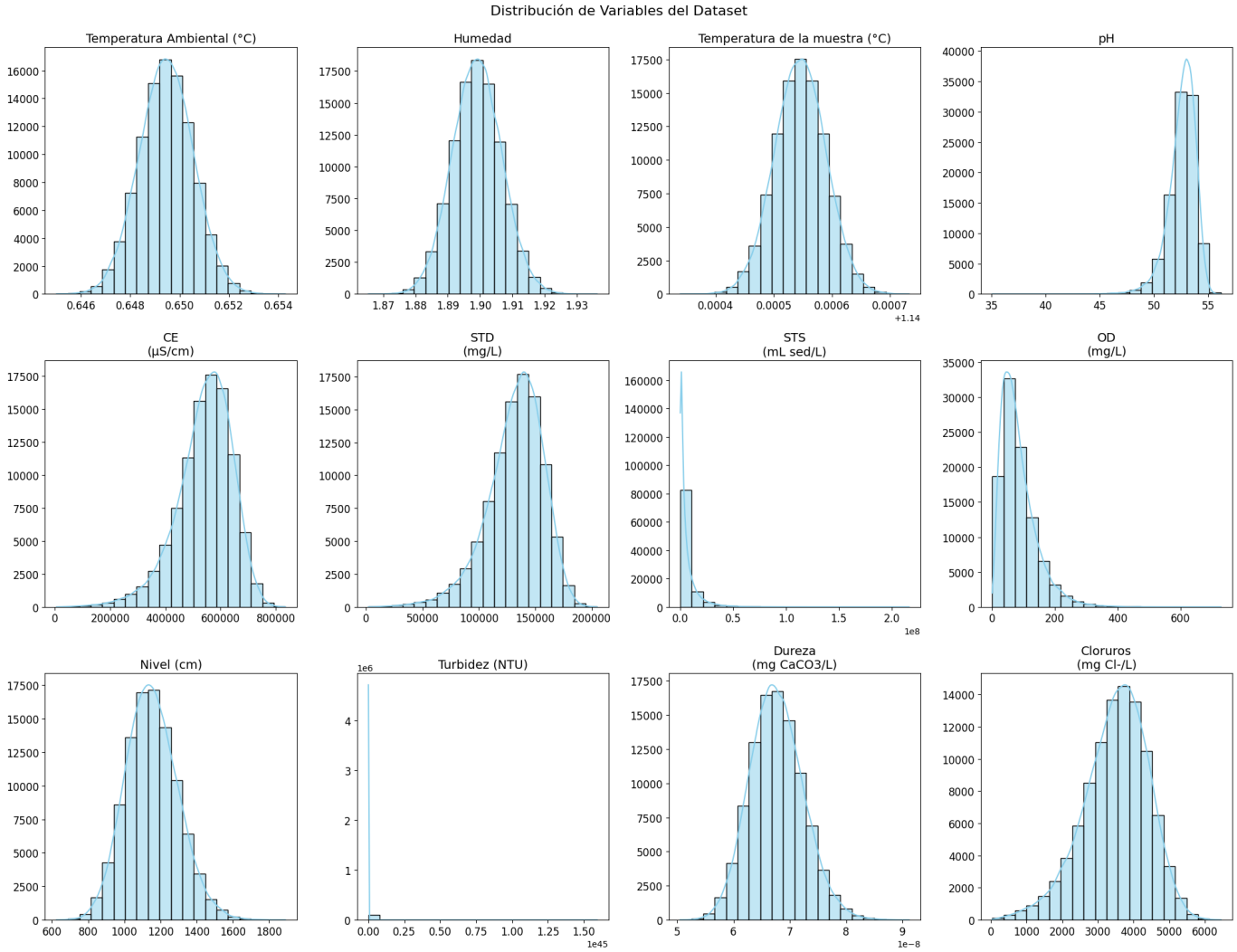
]

Esta transformación inversa se aplica a los datos simulados para obtener un nuevo conjunto de datos que tiene las mismas propiedades estadísticas que el conjunto de datos original pero con la varianza estabilizada y una forma más cercana a la normalidad.

Simulación 6.1. , Transformación de Yeo-Johnson

In [13]:
```
#import pandas as pd
#import numpy as np
#from scipy.stats import yeojohnson

# Copia del DataFrame sin nulos para la simulación
df_original = df_sin_nulos.copy().astype('float64')

# Creamos un DataFrame vacío para guardar los datos transformados con Yeo-Johnson
df_simulado_yeojohnson = pd.DataFrame()

# Generamos datos simulados para cada columna usando la transformación de Yeo-Johnson
for columna in df_original.columns:

    # Eliminamos los ceros para evitar problemas con la transformación de Yeo-Johnson
    datos_sin_cero = df_original[df_original[columna] > 0][columna]

    if len(datos_sin_cero) > 0:
        # Aplicamos la transformación de Yeo-Johnson
        datos_transformados, valor_lambda = yeojohnson(datos_sin_cero)

        # Calculamos la media y la desviación estándar de los datos transformados
        media_transformada = np.mean(datos_transformados)
        desviacion_transformada = np.std(datos_transformados)

        # Generamos números aleatorios con distribución normal
        datos_normales = np.random.normal(media_transformada, desviacion_transformada

        # Aplicamos la transformación inversa de Yeo-Johnson
        if valor_lambda == 0:
            datos_simulados = np.exp(datos_normales)
        else:
            datos_simulados = ((datos_normales * desviacion_transformada) + media_tra
            #datos_simulados = (((datos_normales * valor_lambda) + 1) ** (valor_lambd


        df_simulado_yeojohnson[columna] = datos_simulados

plot_results(df_simulado_yeojohnson)
df_simulado_yeojohnson.to_csv("./dataset_simulado_yeojohnson.csv", index=False)
```

```
/tmp/ipykernel_6688/658340990.py:32: RuntimeWarning: invalid value encountered in powe
r
  datos_simulados = ((datos_normales * desviacion_transformada) + media_transformada)
** (1 / valor_lambda)
/tmp/ipykernel_6688/658340990.py:32: RuntimeWarning: invalid value encountered in powe
r
  datos_simulados = ((datos_normales * desviacion_transformada) + media_transformada)
** (1 / valor_lambda)
/tmp/ipykernel_6688/658340990.py:32: RuntimeWarning: invalid value encountered in powe
r
  datos_simulados = ((datos_normales * desviacion_transformada) + media_transformada)
** (1 / valor_lambda)
/tmp/ipykernel_6688/658340990.py:32: RuntimeWarning: invalid value encountered in powe
r
  datos_simulados = ((datos_normales * desviacion_transformada) + media_transformada)
** (1 / valor_lambda)
/tmp/ipykernel_6688/658340990.py:32: RuntimeWarning: invalid value encountered in powe
r
  datos_simulados = ((datos_normales * desviacion_transformada) + media_transformada)
** (1 / valor_lambda)
```



Explicación de la transformación Yeo-Johnson

La transformación de Yeo-Johnson es otra técnica de transformación que es similar a la transformación de Box-Cox, pero tiene la ventaja de que puede manejar datos que contienen ceros y valores negativos, lo cual es una limitación de la transformación de Box-Cox. Esta técnica de transformación que puede ayudar a que los datos se ajusten mejor a una distribución normal, lo que facilita la simulación de Montecarlo. La fórmula general de la transformación para un conjunto de datos (x) y un parámetro (\lambda) es la siguiente:

[

y(\lambda) =

$$\begin{cases} (y+1)^{\lambda} - 1 & \text{si } \lambda \neq 0, y \geq 0 \\ \ln(y+1) & \text{si } \lambda = 0, y \geq 0 \\ (-y+1)^{\lambda} - 1 & \text{si } \lambda \neq 0, y < 0 \\ \ln(-y+1) & \text{si } \lambda = 0, y < 0 \end{cases}$$

]

Simulación de Datos

1. **Datos originales sin ceros**: Primero, se eliminan los ceros del conjunto de datos original para cada columna, ya que la transformación Box-Cox no se define para (x \leq 0).

2. **Aplicar Yeo-Johnson**: Luego, se aplica la transformación a estos datos para obtener un conjunto transformado. Esta transformación se hace para que los datos transformados se ajusten mejor a una distribución normal.

3. **Calcular estadísticas**: Se calculan la media y la desviación estándar de los datos transformados.

4. **Generar datos aleatorios**: Se generan datos aleatorios con una distribución normal usando la media y la desviación estándar calculadas. Estos datos son una simulación de los datos transformados.

Transformación Inversa de Yeo-Johnson

Una vez que se tienen los datos simulados, se aplica la transformación inversa de Yeo-Johnson para llevarlos de vuelta al espacio original. La fórmula de la transformación inversa es:

[x(y) =

$$\begin{cases} (e^{y} - 1) & \text{si } \lambda = 0 \\ (((\lambda y + 1)^{\lambda} - 1)/\lambda) & \text{si } \lambda \neq 0 \end{cases}$$

]

Esta transformación inversa se aplica a los datos simulados para obtener un nuevo conjunto de datos que tiene las mismas propiedades estadísticas que el conjunto de datos original pero con la varianza estabilizada y una forma más cercana a la normalidad.